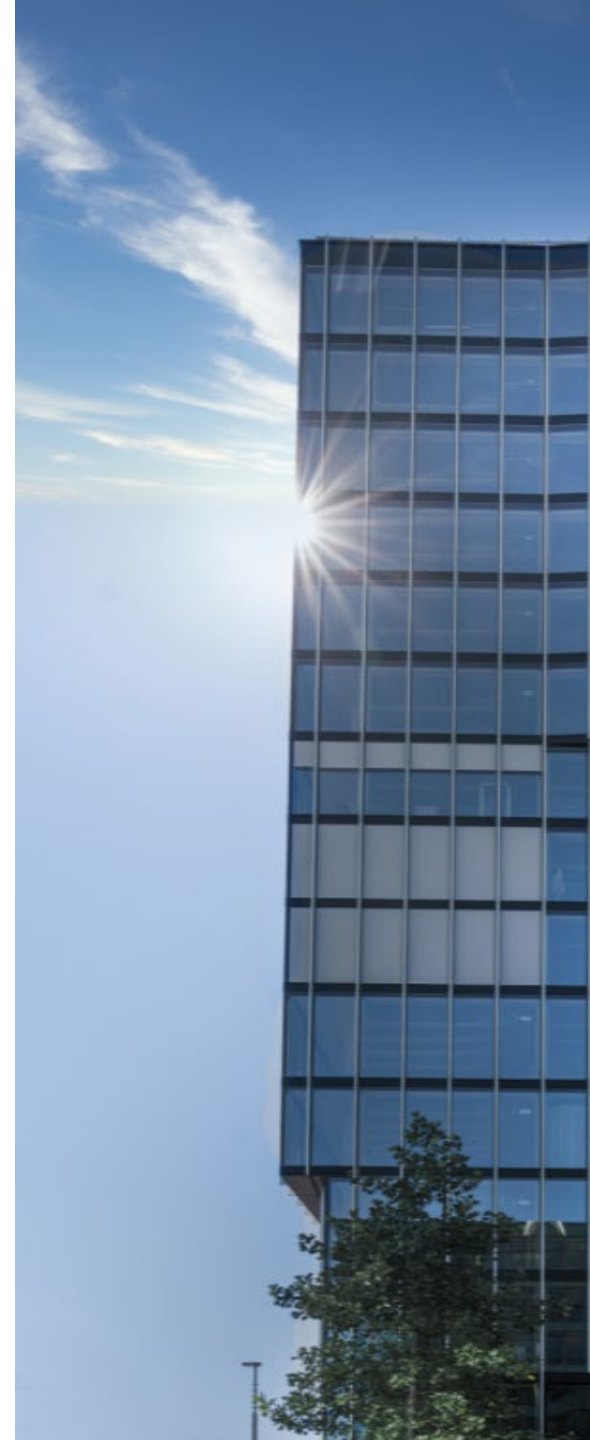


Objektorientierte Programmierung

# Lambdas - Funktionale Programmierung

Roland Gisler



# Inhalt

- Beispiel
- Programmier-Paradigmen
- Lambda-Expression
- Functional Interfaces
- Beispiele: Anwendung von Lambda-Expressions
- Zusammenfassung

# Lernziele

- Sie können den Begriff «Funktionale Programmierung» einordnen.
- Sie wissen, was eine Lambda-Expression ist.
- Sie wissen, was ein «Functional Interface» ist.
- Sie können in Java einfache Lambda-Expressions implementieren und anwenden.

# Beispiel

## Beispiel – Anonyme innere Klasse für Eventhandling

- Sie erinnern sich an den Input O12 Eventhandling mit Hilfe von **anonymen** inneren Klassen (in der Klasse **Auto**):

```
...
final Motor motor = new Motor();
this.motor.addPropertyChangeListener(new PropertyChangeListener() {

    @Override
    public void propertyChange(final PropertyChangeEvent event) {
        LOG.info("Motor-Listener wurde informiert!");
    }
});
...
```

- Wie funktioniert das nochmal ganz genau?

## Beispiel - Behandlung von Events in Java

- In OO-Sprachen wie Java übergeben wir als «Listener» Objekte.
  - Zwecks loser Kopplung implementieren diese Objekte ein spezifisches **Interface**.
  - Häufig (aber nicht immer) enthalten diese Interfaces nur genau **eine** (abstrakte) Methode, die wir implementieren müssen!
- Beispiel des **PropertyChangeListener**-Interface, Quellcode aus dem Modul `java.desktop`, Package `java.beans`:

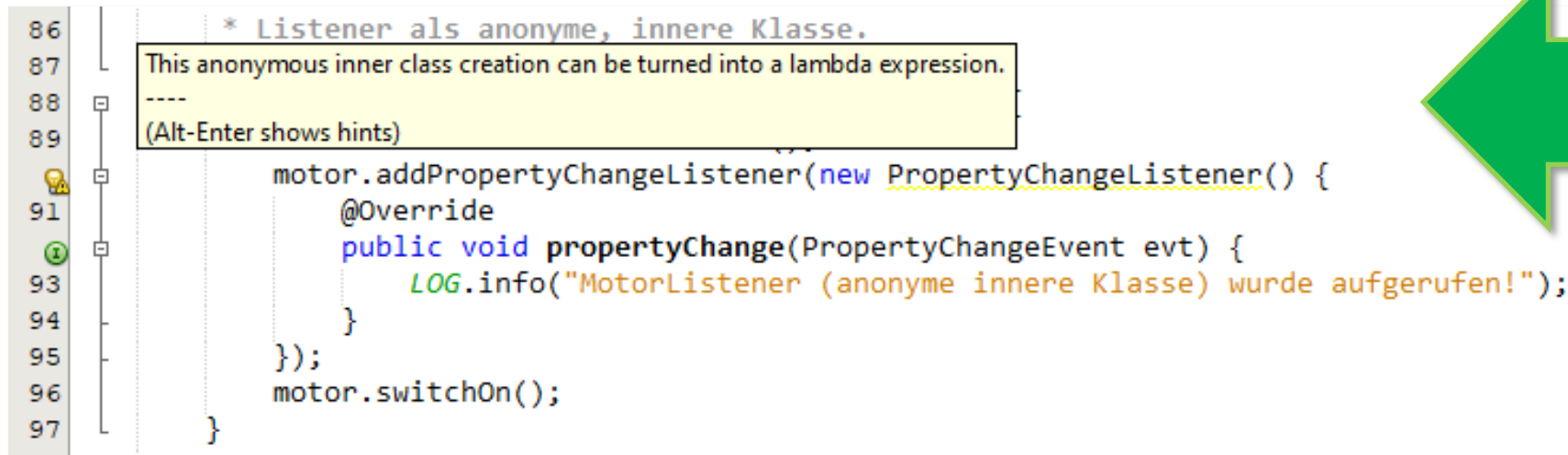
```
interface PropertyChangeListener extends java.util.EventListener {  
    void propertyChange(PropertyChangeEvent evt);  
}
```

Copyright (c) 1996, Oracle and/or its affiliates. All rights reserved.

- Das ist zwar objektorientiert, aber mit relativ viel Aufwand und Code verbunden. Ginge das nicht einfacher?

## Beispiel - Refactoring-Vorschlag der IDE

- Tatsächlich schlagen uns die IDEs vor, den Code von anonymen inneren Klassen in eine «Lambda-Expression» zu verwandeln.
- Am Beispiel des **PropertyChangeListener** von oben:



```
86      * Listener als anonyme, innere Klasse.  
87  
88      This anonymous inner class creation can be turned into a lambda expression.  
89      ----  
90      (Alt-Enter shows hints)  
91  
92      motor.addPropertyChangeListener(new PropertyChangeListener() {  
93          @Override  
94          public void propertyChange(PropertyChangeEvent evt) {  
95              LOG.info("MotorListener (anonyme innere Klasse) wurde aufgerufen!");  
96          }  
97      });  
98      motor.switchOn();  
99  }
```

- Was kommt dabei heraus? Und was ist eine Lambda-Expression?
  - Das klären wir (später, bitte noch etwas Geduld 😊) in diesem Input!

# Programmier-Paradigmen



# Strukturiert/Prozedural und Objektorientiert

- **Strukturierte & Prozedurale Programmierung (SP)**
  - Daten werden in → Datentypen und –strukturen gespeichert.
  - Funktionen und Prozeduren sind in Libraries zusammengefasst.
  - Funktionen und Prozeduren verarbeiten diese Daten.
  - Daten und Funktionen sind organisatorisch eher getrennt.
  - Beispiele von Sprachen: C, Pascal, Modula, ...
- **Objektorientierte Programmierung (OOP)**
  - Wir haben die gleichen Möglichkeiten wie bei **SP**, die Daten und Methoden werden aber (soweit sinnvoll) in Objekten/Klassen zusammengefasst.
  - Potenzial der → Datenkapselung und → Information Hiding.
  - Beispiele: Java, C++, Smalltalk, ...
- Beides wird mehrheitlich mit **imperativen Sprachen** umgesetzt.

## OOP: «Asymmetrie» zwischen Daten und Methoden

- In vielen OO-Sprachen können wir mit «reinen» Datenwerten (Values, primitive Datentypen) arbeiten, sie also als Variablen, Parameter, Attribute etc. an verschiedenen Stellen verwenden.
- Gleiches geht aber **nicht** mit Methoden (Funktionen):  
Diese müssen **immer** einer Klasse zugeordnet sein!
- Aber es gäbe doch auch «reine» Funktionen, die völlig unabhängig sind! Beispiel:  
`add(s1, s2)` → Addiert zwei Zahlen.
  - Diese muss (weil zustandlos) eigentlich keinem Objekt/Klasse gehören!
- Warum sollten wir nicht auch Funktionen «freistehend» verwenden können, und sie z.B. als aktuelle Parameter (!) an andere (höhere) Funktionen übergeben, oder sie in Variablen speichern können?

➔ **Funktionale Programmierung!**

# Paradigma: Funktionale Programmierung (FP)

- In der funktionalen Programmierung können **Funktionen** (Methoden, Prozeduren) nicht nur definiert und aufgerufen, sondern auch:
  - als Parameter (für andere Funktionen) dienen.
  - als Rückgabe (von anderen Funktionen) geliefert werden.
  - sogar miteinander verknüpft werden.
- Funktionen können also wie Daten und Objekte genutzt werden!
- Das Paradigma der funktionalen Programmierung ermöglicht als Alternative eine eher **deklarative Programmierung**.
- In Java 8 wurden Teile der funktionalen Programmierung mit der Einführung von so genannten → **Lambda**-Expressions realisiert.
- Seit Version 8 ist Java somit eine **hybride Sprache** (OOP und FP).

# Funktionale Programmierung in Java

- Mit den Lambda-Expressions schlägt Java eine Brücke von der imperativen, **strukturierten** und **objektorientierten** Programmierung hin zur eher deklarativen, **funktionalen** Programmierung.
  - Die identischen Algorithmen können in Java nun also sowohl (klassisch) strukturiert und **imperativ**, als auch mit Lambdas funktional und **deklarativ** umgesetzt werden!
- Der deklarative Ansatz ist speziell bei der Verarbeitung von Daten als Datenströme mittels **→Streams** sehr attraktiv.
  - Spoiler-Alarm: Folgt im zweiten Input! 😊
- ➔ Mehr zum Thema Programmier-Paradigmen:  
Wahlmodul **PCP** (**P**rogramming **C**oncepts and **P**aradigms).

# Lambda-Expressions

## Beispiel – Lambda Expression

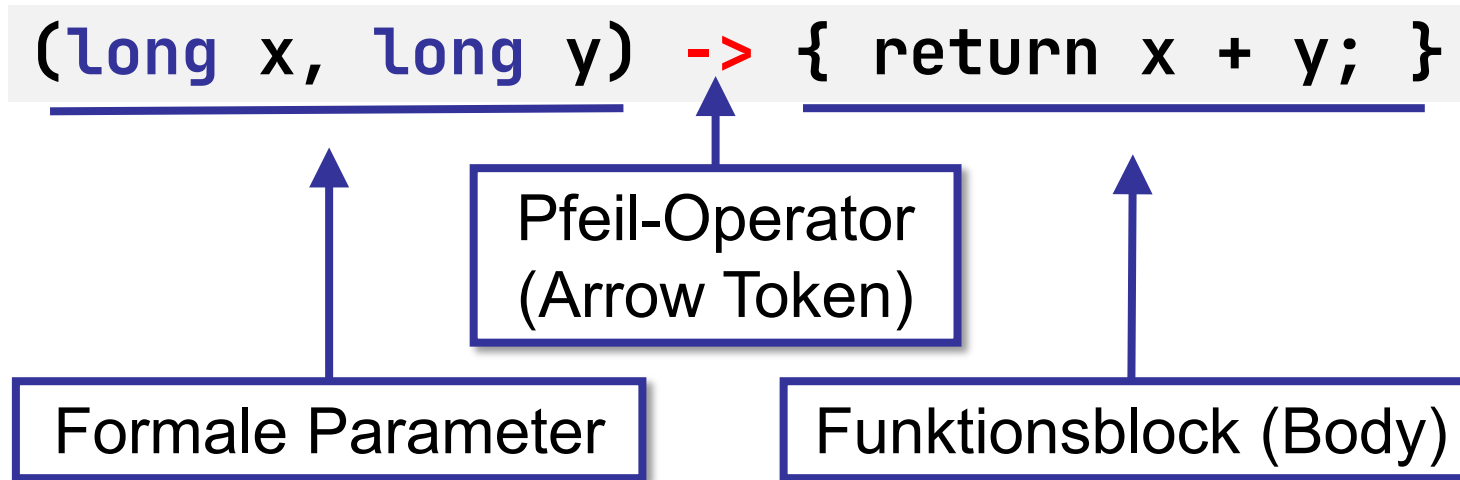
- Hier nun das Resultat des Refactorings des **PropertyChange-Listeners** von oben zu einer Lambda-Expression :

```
this.motor.addPropertyChangeListener(  
    (PropertyChangeEvent event) -> {  
        LOG.info("MotorListener als Lambda implementiert!");  
    }  
);
```

- Im Vergleich stellen wir fest: Der Typ des Interfaces, der Klassenrumpf und sogar der Methodename sind einfach entfallen!
- Wir haben nur noch die Argumentliste "(...)" der nun «anonymen» Funktion, den Pfeil-Operator "->" und die Implementation {...} der (einzigen) Methode des Interfaces, das ➔Lambda!

# Aufbau und Syntax einer Lambda-Expression

- Eine Lambda-Expression ist in Java wie folgt aufgebaut (am Beispiel für ein Lambda zur Addition von zwei **long**-Zahlen):



- Das Lambda ist wie eine Funktion **ohne** Namen, man könnte somit von einer «**anonymen Funktion**» (kein offizieller Begriff) sprechen.
- **Wichtig:** Lambda-Expressions machen in der Regel nur dann wirklich Sinn, wenn Sie wirklich **sehr kurz** und **kompakt** sind!
  - Mehrzeilige Blöcke werden unübersichtlich und sind verpönt!

# Lambda-Expression – Vereinfachungen – 1

- Damit die Lambda-Expressions möglichst kompakt werden, können diese sogar noch weiter vereinfacht werden!
- Die Typinformationen der Parameter können z.B. entfallen:

```
(x, y) -> { return x + y; }
```

- Dadurch wird das Lambda gleichzeitig auch polymorph: Es wird für alle kompatiblen Typen nutzbar, welche die Additions-Operation (+) kennen.
- Besteht die Implementation aus **nur aus einem Ausdruck**, können die geschweiften Klammern, das **return**-Statement und auch das Semikolon (;) entfallen:

```
(x, y) -> x + y
```

- ➔ Der Ausdruck wird automatisch als Rückgabewert verwendet.
- ➔ Diese Variante ist die **kürzest** mögliche Form eines Lambdas.



## Lambda-Expression – Vereinfachungen – 2

- Besitzt eine Lambda-Expression nur einen **einzigen** formalen Parameter, wie z.B. hier:

```
(int x) -> { return x * 2; }
```

können sogar auch die Klammern um den Parameter entfallen:

```
x -> x * 2
```

- Hat ein Lambda jedoch **keine** Parameter, müssen die leeren Parameterklammern zwingend vorhanden sein:

```
() -> this.doSomething()
```

- Wenn die Parameter des Lambdas mit einer darin aufgerufenen, einzigen Methode korrelieren, kann man schliesslich maximal abkürzen (Details folgen):

```
this::doSomething            oder            Klasse::doSomething
```

## Lambda-Expression – Methodenreferenz

- Wenn wir als Aktion «nur» **einen** einzigen Methodenaufruf machen wollen, können wir so genannte **Methodenreferenzen** mit `<objekt/klasse>::<methode>` nutzen (**ohne** Klammern!).
- Notwendige Bedingung: Auf dem Objekt (oder der Klasse) **muss** eine entsprechend benannte (statische) Methode existieren, deren **Parameterliste** mit derjenigen des Lambdas **übereinstimmt** bzw. **typkompatibel** ist (Overloads sind dabei erlaubt!)
- Beispiel für einen **PropertyChangeListener**, welcher in seiner Methode einen **PropertyChangeEvent** als Parameter erwartet:

```
this.motor.addPropertyChangeListener(this::handle);
```

- Notwendige Bedingung: Auf **this** (hier: dem **Auto**) muss eine Methode **handle(PropertyChangeEvent e)** vorhanden sein (Parameter!).
- ➔ Syntax kann anfangs verwirren, weil sie erfordert **viel implizites** Wissen!

# **Funktionsweise von Lambdas in Java**

# Funktionsweise von Lambdas – Functional Interfaces

- Lambdas werden in Java über «**functional interfaces**» realisiert, welche nur eine **einzig**e abstrakte Methode enthalten dürfen.
  - «Zufälligerweise» gab und gibt es in Java schon sehr viele Interfaces, welche dieses Kriterium erfüllen! Siehe z.B. den **PropertyChangeListener** von oben!
- Früher wurden solche Interfaces **SAMs** (**S**ingle **A**bstract **M**ethode) genannt, heute bezeichnet man sie als so genannte **Functional Interfaces**.
- Hinter **jedem** Lambda steht (muss!) in Java somit eines der bereits zahlreich vorhandenen «funktional interfaces» **als Typ**!
- Und tatsächlich werden Lambdas vom Java Compiler wieder in ganz «normale» Objekte übersetzt, welche das jeweilige Interfaces implementieren!
  - Gleichzeitig der Preis und das Potential der (gewollten) Rückwärtskompatibilität.
  - Tatsächlich sind Lambdas im Java Typsystem vollständig rückwärtskompatibel!

# Functional Interfaces

- Da Lambdas ja keine Namen haben, unterscheiden sie sich «nur» in der Anzahl und Reihenfolge der Parameter sowie des Rückgabetyps.
- Da Java ja zusätzlich **generisch** ist, konnten somit häufig verwendete Kombinationen von Parameter- und Rückgabetypen als generische Interfaces mit allgemeinen Namen vordefiniert werden!
  - Diese sind im Package **java.util.function** zu finden!
  - siehe JavaDoc [/java/util/function/package-summary.html](https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html)

Einige ausgewählte Beispiele:

- **UnaryOperator<T>** **Ein** Argument und Return, alle vom Typ **T**.
- **BinaryOperator<T>** **Zwei** Argumente und Return, alle vom Typ **T**.
- **BiFunction<T, U, R>** **Zwei** unterschiedliche Argumente (**T, U**) und Return **R**.
- **Predicate<T>** **Ein** Argument (**T**), Return **boolean**.

# Einsatz von Lambdas in Java

- Lambdas sind die etwas spezielle technische Umsetzung (mit Interfaces) des Ansatzes der funktionalen Programmierung für Java.
- Der Einsatz von funktionaler Programmierung ist in Java aber **kein Zwang**, sondern ein **optionaler, alternativer** Weg.
- Das ist Vor- **und** Nachteil gleichzeitig: In vielen Situationen gibt es nun (noch mehr) **verschiedene** Lösungsansätze.
  - Vergleichbar mit **iterative** versus **rekursive** Lösungen.
- Lambdas führen typischerweise zu markant weniger Quellcode.
- **Dilemma**: Was ist nun die jeweils «beste» Lösung? Und welche Lösung ist am leichtesten verständlich?
- Wichtig: Nicht eine minimale Codemenge ist das wichtigste Kriterium, sondern eine **hohe Ausdrucksstärke!**

## **Beispiele: Anwendung von Lambdas**

## Beispiel 1 - Event-Handling mit Lambdas

- Sie erinnern sich an die Klasse **Motor**, welche mittels eines **PropertyChangeListener**s observiert werden konnte (OOP11):

```
motor.addPropertyChangeListener(new PropertyChangeListener() {  
    @Override  
    public void propertyChange(final PropertyChangeEvent evt) {  
        LOG.info(evt);  
    }  
});
```

- Das lässt sich mit einer Lambda-Expression extrem verkürzen:

```
motor.addPropertyChangeListener(LOG::info);
```

- Die einzig vorhandene Methode **muss** die **propertyChange(...)** sein.
- Sie nimmt einen Parameter **PropertyChangeEvent** entgegen.
- Der wird der **passenden** **LOG.info(Object)**-Methode übergeben, welche dann wiederum auf Objekten automatisch die **toString()**-Methode aufrufen kann.



## Beispiel 2 - Comparator<T>

- Auch die **Comparator<T>**-Schnittstelle ist ein Functional Interface.
  - Zur Erinnerung: In O09 haben wir **Personen**-Objekte mittels eines **Comparators** nach **name** und **vorname** sortiert.
- Da wir damals anonyme innere Klassen noch nicht kannten, hatten wir dafür sogar eine **eigenständige** Klasse implementiert:

```
public final class PersonComparator implements Comparator<Person> {  
  
    @Override  
    public int compare(final Person p1, final Person p2) {  
        int compare = p1.getNachname().compareTo(p2.getNachname());  
        if (compare == 0) {  
            compare = p1.getVorname().compareTo(p2.getVorname());  
        }  
        return compare;  
    }  
}
```

## Beispiel 2 - Comparator<T> - Nun elegant mit Lambdas gelöst!

- Wir können Lambdas nämlich auch in Variablen speichern:

```
final Comparator<Person> nameComp =  
    (p1, p2) -> p1.getNachname().compareTo(p2.getNachname());
```

```
final Comparator<Person> vornameComp =  
    (p1, p2) -> p1.getVorname().compareTo(p2.getVorname());
```

- Und diese können dann (auch «reversed» oder dynamisch kombiniert) zur Sortierung verwendet werden:

```
Collections.sort(persList, nameComp);  
Collections.sort(persList, nameComp.reversed());  
Collections.sort(persList, nameComp.thenComparing(vornameComp));
```

- Quizfrage: `nameComp` ist ja «nur» ein Interface `Comparator<T>`. Was bedeutet es, dass wir darauf eine Methode `reversed()` aufrufen können?

## Beispiel 3 - Ausführen eines Lambdas

- Zuerst definieren wir ein einfaches Lambda (als **BinaryOperator**):

```
final BinaryOperator<Integer> sumLambda = (s1, s2) -> s1 + s2;
```

- Zwei Parameter und Rückgabewert alle vom Typ **T**.

- Dann führen wird das Lambda aus:

```
final int summe = sumLambda.apply(23, 17);
```

- **summe** = Resultat von  $23 + 17 = 40$ .

- Somit ist konzeptionell klar: Die einzige Methode im (Functional-) Interface **BinaryOperator** muss wie folgte aussehen:

```
■ T apply(T p1, T p2)
```

Die Realität ist etwas komplizierter\*, aber wir vereinfachen hier mal wieder. 😊

\* **int** ist ein elementarer Datentyp, darum findet ein Autoboxing zu Integer statt.

# Empfehlungen



- Es dauert einen Moment, bis man die Lambdas versteht. Lassen Sie sich von der IDE helfen, und probieren Sie es aus!
  - Es besteht **kein** Zwang dazu, Lambdas einzusetzen!
  - Mittels Lambdas kann der Quellcode teilweise aber **massiv** verkürzt werden.
- Beachten Sie aber: **Nicht** die **reine Menge** des Quellcodes, **sondern** die hohe **Ausdrucksstärke** soll **entscheidend** sein!
  - Mittelfristig sind Lambdas aber dennoch an vielen Stellen zur deutlichen Verkürzung und Vereinfachung des Quellcodes empfohlen!
- Der sinnvolle Einsatz ist einmal mehr eine Frage des **Programmierstils**.
  - Es gibt nun einfach **noch mehr** verschiedene Lösungswege!
- Zwei (kurze) Tutorials bzw. Quickstarts von Oracle als Empfehlung:  
<http://www.oracle.com/webfolder/technetwork/tutorials/obe/java/Lambda-QuickStart/index.html>  
<https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>



# Zusammenfassung

- Mit Java 8 wurde eine Brücke zwischen der imperativen, objektorientierten Programmierung, und der deklarativen, funktionalen Programmierung geschlagen.  
- **Java ist eine hybride Sprache!**
- Java kennt nun «einen Datentyp» für Funktionen, realisiert über die «Functional Interfaces» (Interfaces mit genau nur **einer** abstrakten Methode).
- Lambdas können den Quellcode beträchtlich verkürzen und den Programmieraufwand verringern, in dem Funktionen sehr kompakt «in-place» definiert werden können.
- Für eine konkrete Anforderung gibt es mit Lambdas nun noch mehr Lösungsvarianten.  
Herausforderung: Welche ist die Beste?



**Fragen?**